

Operating Systems (CS2006)

Date: 24th Feb. 2026

Course Instructor(s)

Dr. Ghufuran Ahmed, Ms. Ansum Hamid,
Mr. Minhaz Raza, Mr. Faisal Ali, Ms. Saeeda, Mr. Zulfiqar, Ms. Rabia,
Mr. UbaidUllah

Sessional-I Exam

Total Time: 1 Hours Total Marks: 15

Total Questions: 02

Semester: Spring 2026

Campus: Karachi

Dept: Computer Science

Student Name

Roll No

Section

Student Signature

Do not use markers or pencils to write your answers

CLO # 1: Describe, discuss, and analyze services provided by modern Operating Systems.

Q1. Attempt answers in the printed order. Do not waste paper. [6 marks, estimated time: 20 minutes]

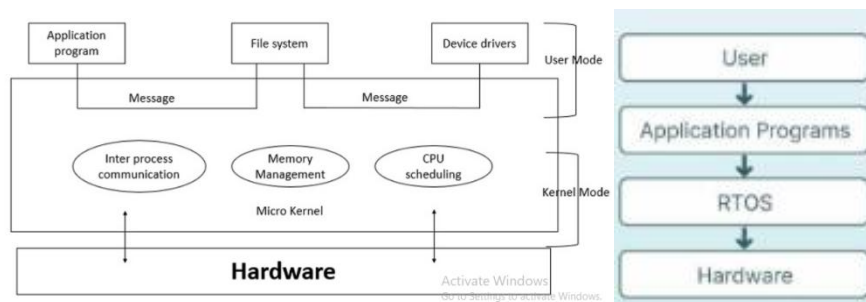
Important Note: Only the first 5 lines of your answers shall be graded.

- a) A centralized system is designed to process large volumes of industrial data in scheduled runs for reporting and analysis, while also responding instantly to critical emergency events such as gas leaks or power failures, where delayed response can cause serious damage. The system integrates frequently updated third-party modules whose failures must not affect core operations, enforces strict separation between user applications and hardware access, and allows individual components to be updated or isolated without shutting down the entire system.

- i. Identify and Sketch the most suitable OS structure and the most suitable OS type (Batch Processing vs Real-Time Processing System). [1]: No code

OS Structure: Microkernel architecture (modular, isolated services)

OS Type: Real-Time Operating System (RTOS)



- ii. Justify your choices and briefly explain why the alternatives are not appropriate. [1]: No code

RTOS is required because the system must respond immediately to emergencies (gas leak, power failure). Batch systems delay execution, so they are unsafe for critical events. Microkernel structure is best because third-party modules can fail without crashing the whole system (fault isolation + reliability). It provides strong separation between user apps and hardware, improving security and stability. Components can be updated or restarted independently without stopping the whole system. Monolithic structure or pure batch OS is not suitable because they are less flexible, less secure, and slower for urgent real-time responses.

“OR”

Dynamic modules (loadable kernel modules) run inside the kernel space, so if one fails, it can crash the entire system. Microkernel keeps most services (file system, network, drivers) in user space, isolating failures from the core kernel. This means a failing module won't crash the whole OS, improving reliability. Microkernel allows independent updating or restarting of components without shutting down the system. So, compared to dynamic modules, microkernel provides better fault isolation, security, and system stability. Same goes for modular monolithic design (hybrid approach). Layered OS organizes services in strict levels, where each layer depends on the one below it. If a lower layer fails, all upper layers may also fail, reducing system reliability. Microkernel allows independent user-space modules, so failures don't crash the entire OS. Layered structure is also less flexible for updating or adding new modules dynamically. Therefore, for a system needing fault isolation, modular updates, and real-time response, microkernel is better than layered.

- b) Now, answer the following: [0.5 +0.5]

- i. When does a process become marked as zombie process. How does OS handle a zombie process? No drawing.

A process becomes a zombie when it terminates but its parent has not yet called wait() to collect its exit status. The OS keeps a small record (PID and exit code) of that process in the process table. When the parent calls wait()/waitpid(), the OS removes the zombie entry completely. If the parent never calls wait, the OS assigns the zombie to init (PID 1) which then cleans it. Thus, a zombie is temporary and automatically removed after its status is collected.

- ii. Explain the difference between wait () and sleep () system call in-terms of generating interrupts between processes in OS.

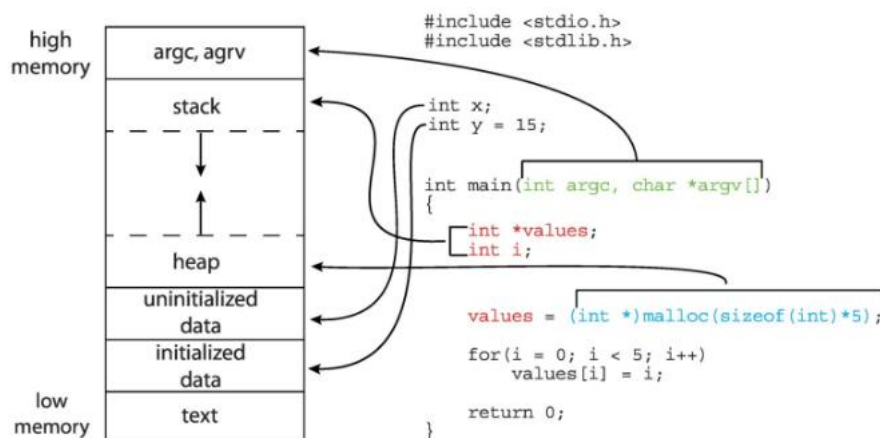
wait() blocks the parent process until a child process terminates. It resumes due to a software interrupt/signal generated by the child's exit. sleep() pauses a process for a fixed time duration. It resumes due to a hardware timer interrupt from the system clock. Hence, wait() is process-event based, while sleep() is time-based.

- c) Why does operating system called a resource allocator? Give a technical reason/example for your decision. No code. No drawing. [1]

The OS is called a resource allocator because it manages hardware resources like CPU, memory, and I/O devices. It decides which process gets CPU time, memory space, or device access at a given moment. For example, CPU scheduling allocates processor time to different processes efficiently. Memory management ensures each process gets safe and sufficient RAM. Without the OS allocating resources, processes would conflict and system crashes could occur.

- d) A process has just been created, and you are the operating system responsible for managing it. Imagine you are the operating system managing a new process. You need to keep track of all information about the process and organize its memory. With the help of diagram Explain how the process related information stored in memory and helps the operating system to manage the process efficiently. [2]

When a process is created, the OS stores its information in a Process Control Block (PCB) in memory. The PCB contains process ID, state, program counter, CPU registers, memory limits, open files, and scheduling info. The OS uses the PCB to track and manage the process, switch CPU between processes, and resume execution correctly. Memory for the process is divided into text (code), data (variables), stack (function calls), and heap (dynamic memory). This organization ensures efficient execution, isolation, and easy context switching.



CLO # 2: Understand, design, and implement solutions employing concepts of Processes/Threads. Q2. Write C snippets in the printed order. [3 marks x 3 parts = 9 marks in total, estimated time: 35 minutes]

Important Note: Avoid missing computational thinking steps, syntax errors, invalid logic, invalid dry run step(s), and input-output statements unless asked. Messy, unreadable, and/or scattered code will get very low scores.

- a) Write a C program where a parent process reads a string from the user and sends it to two child processes using separate pipes:

Child 1 converts all lowercase letters to uppercase. Child 2 reverses the string.

Both children send their results back to the parent using their own pipes, and the parent prints both processed strings. Use fork (), pipe (), read (), and write () system calls. [3]

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
```

```
#define MAX 100
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc != 2) {
        printf("Usage: %s <string>\n", argv[0]);
        return 1;
    }
```

```
    int pipe1[2], pipe2[2]; // Parent -> children
    int pipe1_back[2], pipe2_back[2]; // Children -> parent
    char *input = argv[1];
```

```

// Create pipes
pipe(pipe1); pipe(pipe2);
pipe(pipe1_back); pipe(pipe2_back);

// Child 1: Uppercase
if (fork() == 0) {
    close(pipe1[1]); // Close write end
    close(pipe1_back[0]); // Close read end

    char s[MAX];
    read(pipe1[0], s, MAX);
    close(pipe1[0]);

    for (int i=0; s[i]; i++)
        s[i] = toupper(s[i]);

    write(pipe1_back[1], s, strlen(s)+1);
    close(pipe1_back[1]);
    exit(0);
}

// Child 2: Reverse
if (fork() == 0) {
    close(pipe2[1]);
    close(pipe2_back[0]);

    char s[MAX];
    read(pipe2[0], s, MAX);
    close(pipe2[0]);

    int len = strlen(s);
    for (int i=0; i<len/2; i++) {
        char temp = s[i];
        s[i] = s[len-1-i];
        s[len-1-i] = temp;
    }

    write(pipe2_back[1], s, strlen(s)+1);
    close(pipe2_back[1]);
    exit(0);
}

// Parent process
close(pipe1[0]); close(pipe2[0]);
close(pipe1_back[1]); close(pipe2_back[1]);

write(pipe1[1], input, strlen(input)+1);
write(pipe2[1], input, strlen(input)+1);
close(pipe1[1]); close(pipe2[1]);

char upper[MAX], reversed[MAX];
read(pipe1_back[0], upper, MAX);
read(pipe2_back[0], reversed, MAX);

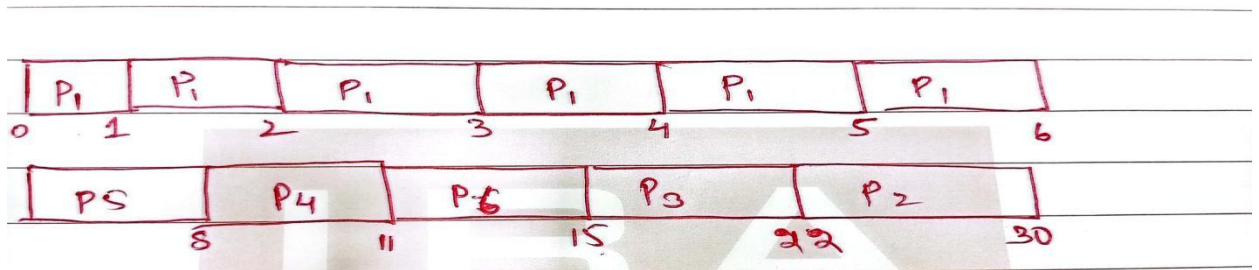
printf("Child 1 (Uppercase): %s\n", upper);
printf("Child 2 (Reversed): %s\n", reversed);

return 0;
}

```

- b) Six processes arrive at different times with varying burst durations as follows: P1(0,6), P2(1,8), P3(2,7), P4(3,3), P5(5,2), P6(6,4), where each ordered pair represents (Arrival Time, Burst Time). Using **Shortest Remaining Time (SRT) scheduling**, simulate the execution of these processes considering **all possible preemptions**. Draw the **Gantt chart** marking every preemption, calculate the **completion time, turnaround time, and waiting time** for each process, identify the **process with minimum waiting time** and explain why it got the CPU first during preemption, and finally discuss how SRT in this scenario helps in **reducing average waiting time compared to FCFS**. [3]

Time 0: Only P1 is available → Run P1 (remaining 6 → 5)
 Time 1: P2 arrives (BT 8) → P1 remaining 5 < P2 BT 8 → P1 continues
 Time 2: P3 arrives (BT 7) → P1 remaining 4 < P3 BT 7 → P1 continues
 Time 3: P4 arrives (BT 3) → Compare remaining times:
 P1 remaining = 3
 P4 BT = 3
 → Tie, assume current process continues → P1 continues
 Time 4: No new arrivals → P1 remaining 2 → continue
 Time 5: P5 arrives (BT 2) → Compare:
 P1 remaining = 1
 P5 BT = 2 → P1 remaining < P5 → P1 continues
 Time 6: P6 arrives (BT 4) → P1 remaining = 0 → P1 finishes
 So first execution: P1 runs from 0–6 with no preemption.
 Now at time 6, available processes: P2(8), P3(7), P4(3), P5(2), P6(4)
 Choose shortest remaining time → P5 (2) → run 6–8 → P5 finishes
 Time 8: Remaining: P2(8), P3(7), P4(3), P6(4) → Shortest = P4(3) → run 8–11 → P4 finishes
 Time 11: Remaining: P2(8), P3(7), P6(4) → Shortest = P6(4) → run 11–15 → P6 finishes
 Time 15: Remaining: P2(8), P3(7) → Shortest = P3(7) → run 15–22 → P3 finishes
 Time 22: Remaining: P2(8) → run 22–30 → P2 finishes



Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	0	6	6	6	0
P2	1	8	30	29	21
P3	2	7	22	20	13
P4	3	3	11	8	5
P5	5	2	8	3	1
P6	6	4	15	9	5

Minimum Waiting Time: $P1 = 0 \rightarrow$ got CPU first because it arrived earliest and no other process was available.

Average Turnaround Time (TAT) = $(6+29+20+8+3+9)/6 = 12.5$

Average Waiting Time (WT) = $(0+21+13+5+1+5)/6 = 7.5$

Shorter processes like P5 and P4 get CPU earlier when they arrive, reducing average waiting time compared to FCFS.

SRT prioritizes processes with the shortest remaining burst time, preventing long processes from blocking shorter ones

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t p1, p2, p3;
    p1 = fork();
    if (p1 == 0) {
        p2 = fork();
        if (p2 == 0) {
            fork();
        }
        fork();
    }
    else {
        p3 = fork();
        if (p3 == 0)
            fork();
    }
    printf("PID=%d, PPID=%d\n", getpid(), getppid());
    return 0;
}
```

- c) Draw the complete process tree showing all parent-child relationships. Count the total number of processes created (including parent). Explain the possible output order if each process prints its PID and PPID. Identify which processes are siblings and which are grandchildren. [3]

```

P0 (original)
├─ P1 (created by Fork #1)
│   └─ P2 (created by Fork #2)
│       └─ P3 (created by Fork #3)
│           └─ P4 (created by Fork #4 executed by P2)
│       └─ P5 (created by Fork #4 executed by P1)
└─ P6 (created by Fork #5)
    └─ P7 (created by Fork #6)

```

Total processes created: 8 (1 original + 7 from 6 fork() calls — note that Fork #4 runs twice, once in P1 and once in P2).

This is the complete hierarchy. No other forks exist, and every process ends after executing its portion of the code.

The kernel scheduler decides the order — completely unpredictable in general No synchronization (no wait(), no pipes, no signals, no sleep) → race condition between all prints . Fork order is fixed, but execution continues in parallel immediately after fork(). Parent and child are both runnable right after fork() Modern systems usually give child a small initial timeslice, but this is not guaranteed.

